ABSTRACT

This project presents the design and implementation of an Advanced Driver Assistance System (ADAS) for Traffic Sign Detection and Classification, aimed at improving road safety through automated perception of traffic signs. The system follows a two-stage pipeline, where the first stage focuses on detecting and extracting the traffic sign region from a full road scene image, and the second stage classifies the detected sign into one of 43 categories using deep learning techniques. The detection module combines traditional computer vision methods such as HSV color thresholding and contour analysis with a deep learning-based fallback mechanism using YOLO, ensuring robustness across varying real-world conditions including illumination changes, occlusions, and background noise. The classification module leverages both a custom Convolutional Neural Network (CNN) and transfer learning models such as MobileNetV2 and ResNet50, enabling a comprehensive comparison of performance across different architectures. A key contribution of this project is the analysis of the Speed vs Accuracy trade-off, which is critical for real-time ADAS deployment. Experimental results demonstrate that while ResNet50 achieves the highest accuracy, MobileNetV2 provides faster inference suitable for edge devices. The final system is deployed as an interactive web application using Streamlit, allowing users to upload road scene images and receive real-time detection and classification results. Overall, this project demonstrates a practical and scalable approach to traffic sign recognition, highlighting the importance of integrating detection and classification for real-world intelligent transportation systems.

CHAPTER 01

INTRODUCTION

With the rapid evolution of intelligent transportation systems, the role of Advanced Driver Assistance Systems (ADAS) has become increasingly significant in enhancing road safety and driving efficiency. ADAS technologies aim to reduce human intervention by assisting drivers in tasks such as lane keeping, collision avoidance, and traffic sign recognition. Among these, traffic sign detection and classification is a critical component because it directly influences driving decisions.

Traffic signs convey essential information about road rules, hazards, and navigation. Human drivers interpret these signs visually; however, replicating this ability in machines requires sophisticated computer vision and deep learning techniques. The challenge lies not only in recognizing the sign but also in accurately detecting it from complex real-world scenes that include noise, varying lighting conditions, occlusions, and motion blur.

This project addresses this challenge by developing a **two-stage pipeline**:

- A **detection stage**, which extracts the region of interest (traffic sign) from a full image
- A **classification stage**, which identifies the category of the detected sign

Unlike simplified classification tasks that rely on pre-cropped datasets, this project simulates a real-world ADAS scenario, where the system must first locate the sign before recognizing it. This significantly increases the complexity and practical relevance of the system.

Furthermore, the project evaluates different deep learning models, including custom CNN, MobileNetV2, and ResNet50, to analyze the trade-offs between computational efficiency and prediction accuracy. This is particularly important in ADAS systems, where real-time performance is critical.

CHAPTER 02

PROBLEM STATEMENT

The primary objective of this project is to design and implement a robust system capable of performing traffic sign detection and classification in real-world conditions. The system must process a full road scene image, identify the location of a traffic sign, and classify it into one of 43 predefined categories from the GTSRB dataset.

2.1 Objectives

The project is structured around the following key objectives:

1. To develop a preprocessing pipeline that standardizes input images for model training and inference
2. To perform exploratory data analysis (EDA) to understand dataset characteristics and challenges
3. To implement a detection algorithm capable of identifying traffic signs in full images
4. To design and train multiple deep learning models for classification
5. To compare models based on accuracy, speed, and computational efficiency
6. To integrate detection and classification into a unified pipeline
7. To deploy the solution using a user-friendly interface.

2.2 Challenges

The project involves several real-world challenges:

1. Illumination Variability

Traffic signs appear differently under varying lighting conditions such as bright sunlight, shadows, fog, or night-time scenarios. These variations affect pixel intensity and color representation, making detection and classification difficult.

2. Motion Blur

Images captured from moving vehicles often contain motion blur, which reduces sharpness and makes it harder to identify edges and symbols.

3. Occlusion

Traffic signs may be partially blocked by trees, vehicles, or other objects. This leads to incomplete visual information, making classification more challenging.

4. Class Imbalance

The dataset contains unequal representation of classes, causing models to become biased toward frequently occurring classes.

5. Similar Visual Features

Certain traffic signs, especially speed limit signs, have very similar visual patterns, making them difficult to distinguish.

6. Detection Complexity

Unlike classification, detection requires identifying the correct region in an image, which is computationally more complex.

CHAPTER 03

DATASET DESCRIPTION

The project utilizes the German Traffic Sign Recognition Benchmark (GTSRB) dataset, which is widely used for training and evaluating traffic sign recognition models.

3.1 Dataset Structure

The dataset is organized into three main components:

Training Set

- Contains images grouped into 43 folders
- Each folder corresponds to a specific class
- Labels are automatically inferred from folder names

Test Set

- Contains images without folder-based labels
- Labels are provided separately in a CSV file

Metadata

- Includes class descriptions and additional information

3.2 Dataset Characteristics

The dataset includes a wide range of real-world conditions:

- Different camera angles
- Varying distances from the sign
- Diverse backgrounds
- Multiple lighting conditions

The images vary in size and must be resized for uniform processing.

3.3 Class Distribution Analysis

A detailed analysis of class distribution reveals that:

- Some classes have thousands of images
- Others have only a few hundred samples

This imbalance can lead to:

- Overfitting on dominant classes
- Poor generalization on minority classes

To address this, data augmentation techniques are applied.

CHAPTER 04

EXPLORATORY DATA ANALYSIS (EDA)

EDA plays a crucial role in understanding the dataset and identifying potential challenges before model training.

4.1 Visual Analysis

By visualizing sample images from different classes, several observations were made:

- Traffic signs appear at different scales
- Backgrounds vary significantly
- Some images contain noise or blur

4.2 Feature Analysis

Traffic signs can be differentiated based on three key features:

Color

Color is one of the most distinctive features:

- Red indicates prohibition or restriction
- Blue indicates mandatory actions
- Yellow indicates warnings

Shape

Shapes provide structural information:

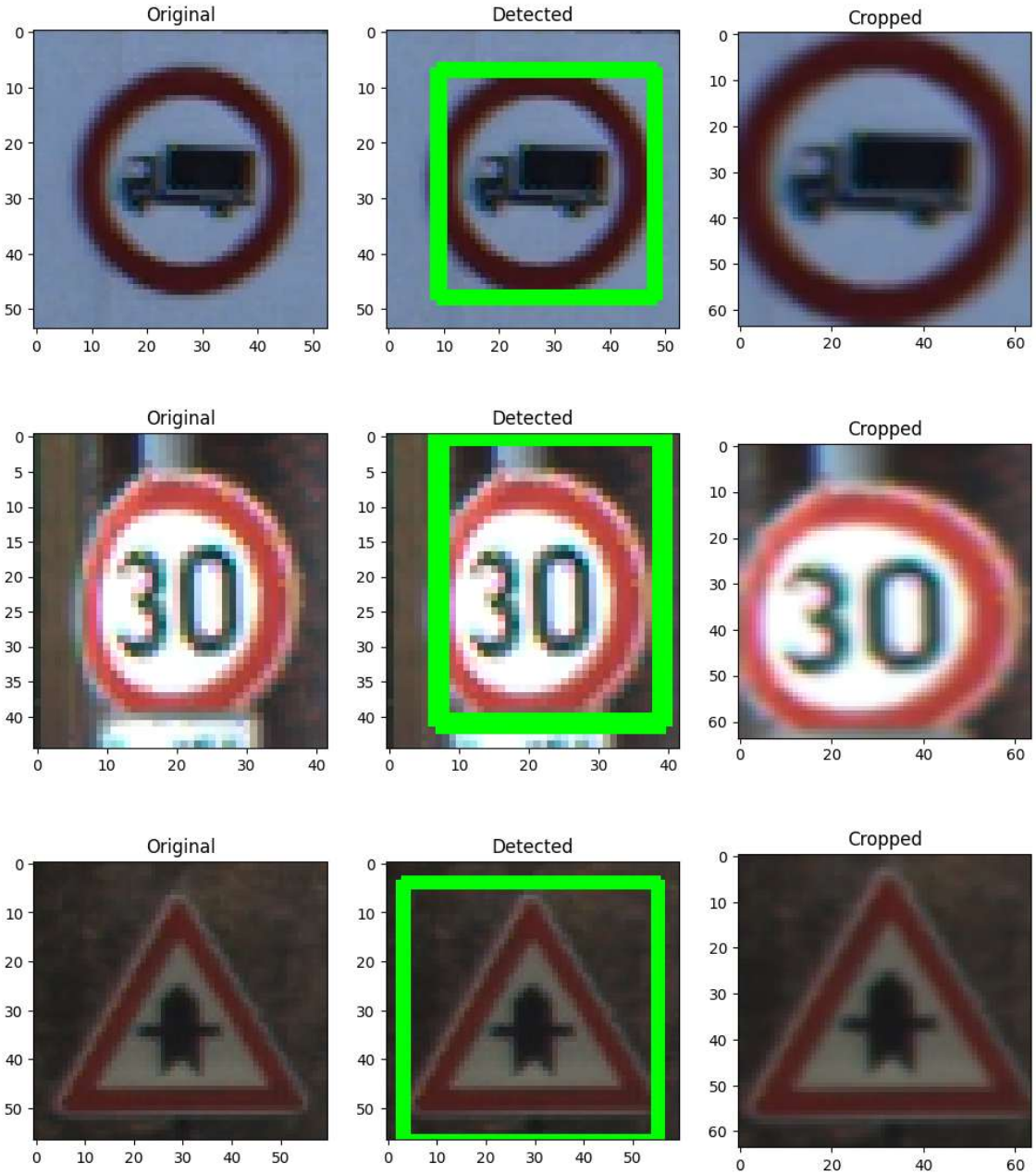
- Circular signs → regulatory
- Triangular signs → warning
- Rectangular signs → informational

Symbol

Symbols inside signs convey specific meanings:

- Numbers for speed limits

- Arrows for direction
- Icons for warnings



CHAPTER 05

DATA PREPROCESSING & AUGMENTATION

5.1 Importance of Preprocessing

Preprocessing ensures that all images are in a consistent format, which is essential for efficient model training. Without preprocessing, models may fail to converge due to inconsistent input dimensions and pixel distributions.

5.2 Preprocessing Steps

1. **Resizing**
   - All images resized to 64×64 pixels
   - Ensures uniform input size
2. **Normalization**
   - Pixel values scaled to [0,1]
   - Helps in faster convergence
3. **Data Splitting**
   - Training set: 80%
   - Validation set: 20%
   - Ensures unbiased evaluation

5.3 Data Augmentation

Data augmentation artificially increases dataset diversity by applying transformations:

Rotation

Simulates different viewing angles

Shifting

Simulates camera movement

Zoom

Simulates distance variation

Brightness Adjustment

Simulates different lighting conditions

Why No Horizontal Flip?

Flipping changes the meaning of signs:

- Left turn becomes right turn
- This leads to incorrect labels

CHAPTER 06

SIGN DETECTION & CROPPING PIPELINE

The detection stage forms the foundation of the entire ADAS pipeline, as the performance of the classification stage is highly dependent on the quality of the detected and cropped traffic sign region. In real-world scenarios, traffic signs are embedded within complex backgrounds that may include roads, vehicles, pedestrians, buildings, and natural elements such as trees and shadows. Therefore, accurately isolating the traffic sign from such environments is a non-trivial task. In this project, a hybrid detection strategy is implemented, combining traditional computer vision techniques with modern deep learning approaches to achieve robust performance across a wide variety of conditions. The primary detection method is based on color segmentation using the HSV color space, which is particularly effective for identifying regions with distinct color characteristics such as traffic signs.

The choice of HSV over RGB is intentional, as HSV separates color information (Hue) from intensity (Value), making it more robust to lighting variations. Traffic signs typically exhibit strong color patterns, such as red borders for prohibitory signs, blue backgrounds for mandatory signs, and yellow backgrounds for warning signs. By defining appropriate thresholds in the HSV space, it becomes possible to isolate these regions effectively.

Multiple color masks are applied to capture different categories of signs. For instance, red masks are defined using two separate ranges to account for the circular nature of the hue spectrum. Similarly, blue and yellow masks are defined to capture mandatory and warning signs respectively. In addition to these, a grayscale threshold is used to detect white regions, which are common in speed limit signs and other regulatory signs.

After generating the combined mask, morphological operations such as closing are applied to remove noise and fill gaps within detected regions. This step is crucial to ensure that the contours extracted in the next stage are clean and representative of actual objects rather than noise artifacts. Contour detection is then performed using OpenCV's contour extraction functions. Each contour represents a potential candidate region that may correspond to a traffic sign. However, not all contours are relevant; therefore, a filtering process is applied. Contours with very small areas are discarded, as they are likely to be noise. Additionally, contours are evaluated based on their aspect ratio, as most traffic signs are approximately square or circular

in shape. This helps eliminate elongated or irregular shapes that do not correspond to traffic signs.

Among the filtered contours, the most suitable candidate is selected based on the largest area, as it is more likely to correspond to the primary traffic sign in the image. A bounding box is then computed around this contour, and the corresponding region is cropped from the original image. To ensure compatibility with the classification model, the cropped region is resized to a fixed dimension of 64×64 pixels. This resizing step is critical, as deep learning models require consistent input dimensions.

Despite the effectiveness of this approach, it has certain limitations. Color-based detection can fail in scenarios where the sign colors are faded, distorted due to lighting, or partially occluded. To address these limitations, a fallback mechanism using YOLO (You Only Look Once) is integrated into the pipeline. YOLO is a deep learning-based object detection algorithm that can identify objects based on learned features rather than predefined color thresholds.

If the OpenCV-based detection fails to identify a valid region, the system automatically switches to YOLO-based detection. YOLO processes the entire image and returns bounding boxes for detected objects along with confidence scores. The most confident detection is selected, and the corresponding region is cropped.

In cases where both OpenCV and YOLO fail, a final fallback strategy is employed, which involves performing a center crop of the image. While this method is not ideal, it ensures that the system always produces an output, thereby improving robustness. Overall, the multi-stage detection pipeline significantly enhances the reliability of the system and ensures that a wide range of real-world scenarios can be handled effectively.

CHAPTER 07

CNN MODEL DEVELOPMENT

The classification stage of the pipeline is responsible for identifying the category of the detected traffic sign. This is achieved using Convolutional Neural Networks (CNNs), which are highly effective for image classification tasks due to their ability to automatically learn hierarchical feature representations.

The CNN architecture used in this project is designed to balance complexity and efficiency. The initial layers of the network focus on extracting low-level features such as edges and textures. As the network depth increases, higher-level features such as shapes and patterns are learned. This hierarchical feature extraction is essential for distinguishing between different traffic sign classes.

The architecture begins with a convolutional layer with 32 filters, followed by a ReLU activation function. This layer captures basic features such as edges and color gradients. A max-pooling layer follows, which reduces the spatial dimensions of the feature maps while retaining the most important information. This helps in reducing computational complexity and preventing overfitting.

A second convolutional layer with 64 filters is then applied, allowing the network to learn more complex features. This is again followed by a max-pooling layer. The output of the convolutional layers is then flattened into a one-dimensional vector, which serves as input to the fully connected layers.

The fully connected layers are responsible for performing the final classification. Multiple dense layers with ReLU activation are used, each followed by dropout layers to prevent overfitting. Dropout works by randomly deactivating a fraction of neurons during training, forcing the network to learn more robust features.

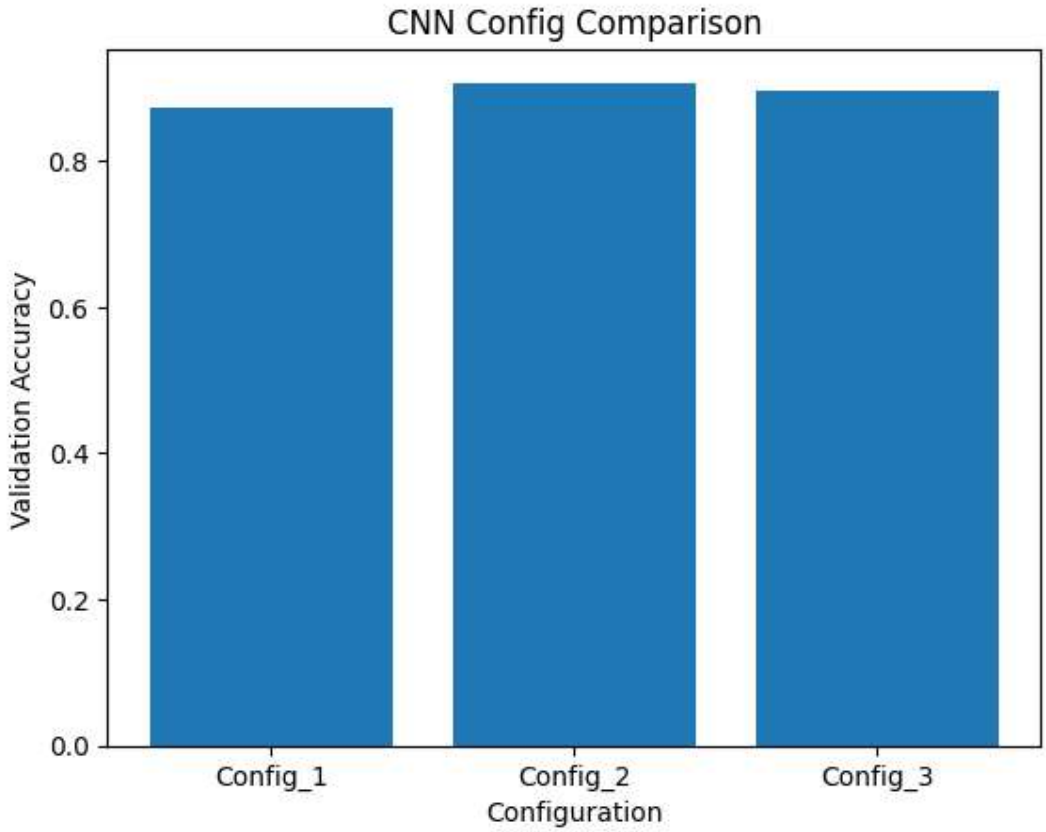
The final output layer uses a softmax activation function, which produces a probability distribution over the 43 classes. The class with the highest probability is selected as the predicted label.

The model is trained using the Adam optimizer, which is widely used due to its adaptive learning rate capabilities. The loss function used is categorical crossentropy, which is suitable for multi-class classification tasks.

To improve training efficiency and prevent overfitting, callbacks such as EarlyStopping and ModelCheckpoint are used. EarlyStopping monitors the validation loss and stops training when it stops improving, while ModelCheckpoint saves the best model based on validation performance.

Multiple configurations of the CNN are experimented with by varying the number of hidden layers and neurons. This helps in identifying the optimal architecture that provides the best balance between accuracy and computational efficiency.

	Config	Hidden Layers	Neurons	Best Val Accuracy	Best Val Loss
0	Config_1	1	128	0.871700	0.446966
1	Config_2	2	256	0.907410	0.370857
2	Config_3	3	512	0.896697	0.409106



CHAPTER 08

TRANSFER LEARNING

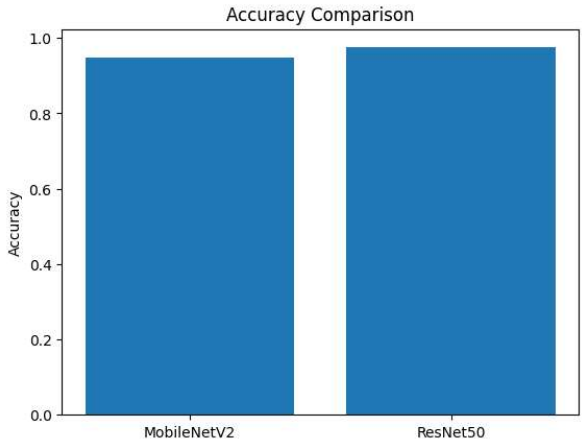
Transfer learning is a powerful technique that leverages pre-trained models to improve performance on a new task. Instead of training a model from scratch, a model that has already been trained on a large dataset (such as ImageNet) is used as a starting point.

In this project, two pre-trained models are used: **MobileNetV2** and **ResNet50**.

- MobileNetV2 is designed for efficiency and is particularly suitable for mobile and embedded devices. It uses depthwise separable convolutions, which significantly reduce the number of parameters and computational cost. This makes it ideal for real-time applications where latency is critical.
- ResNet50, on the other hand, is a deeper and more complex model that uses residual connections to address the vanishing gradient problem. These residual connections allow gradients to flow more easily through the network, enabling the training of very deep architectures.

For both models, the top layers are removed, and a custom classification head is added. The base model is initially frozen to preserve the learned features, and only the new layers are trained. This approach reduces training time and prevents overfitting.

Transfer learning significantly improves the performance of the model, as it allows the network to leverage features learned from a large and diverse dataset.

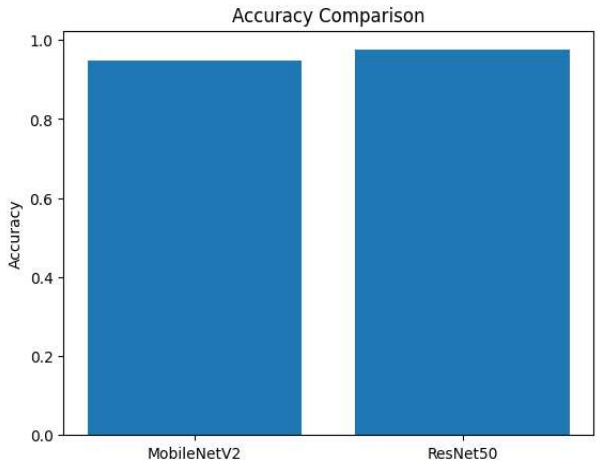


CHAPTER 09

SPEED VS ACCURACY TRADE-OFF

- In ADAS systems, achieving high accuracy is important, but it must not come at the cost of excessive latency. The system must be able to process images in real time to provide timely assistance to the driver.
- The trade-off between speed and accuracy is analyzed by comparing the performance of the custom CNN, MobileNetV2, and ResNet50 models. While ResNet50 provides the highest accuracy, it is also computationally more expensive. MobileNetV2 offers a good balance, with slightly lower accuracy but significantly faster inference.
- The choice of model depends on the deployment environment. For edge devices with limited computational resources, MobileNetV2 may be preferred. For high-performance systems where accuracy is critical, ResNet50 is more suitable.

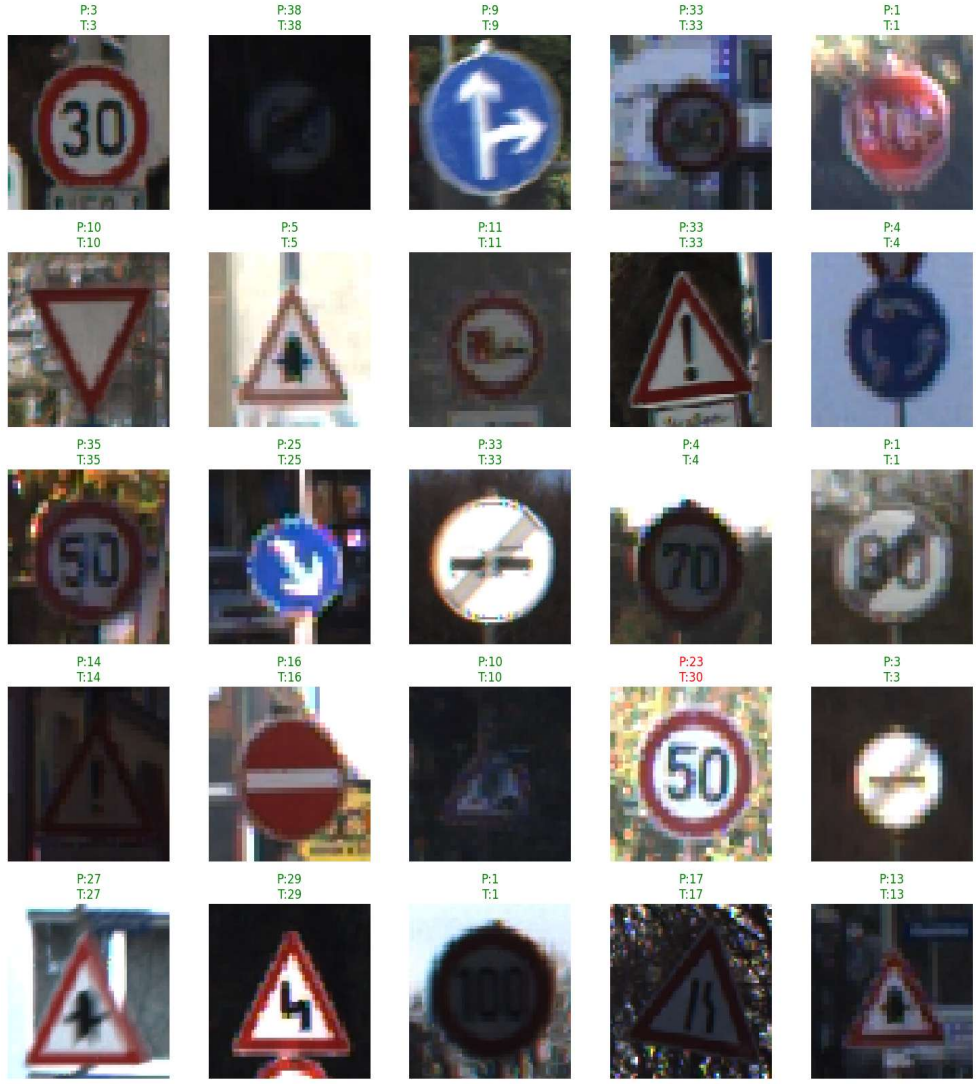
	Model	Val_accuracy	Training Time(min)	Model Size (MB)	Inference Time (ms)
0	MobileNetV2	0.948348	71.384340	28.189621	56.564898
1	Resnet50	0.974876	193.592043	273.376272	32.267983



CHAPTER 10

MODEL EVALUATION

- Model evaluation is performed using multiple metrics to provide a comprehensive understanding of performance.
- Accuracy provides an overall measure of correctness, but it does not capture class-wise performance. Therefore, precision, recall, and F1-score are also computed for each class.
- A confusion matrix is generated to visualize the performance across different classes. This helps in identifying patterns of misclassification and understanding which classes are most challenging.

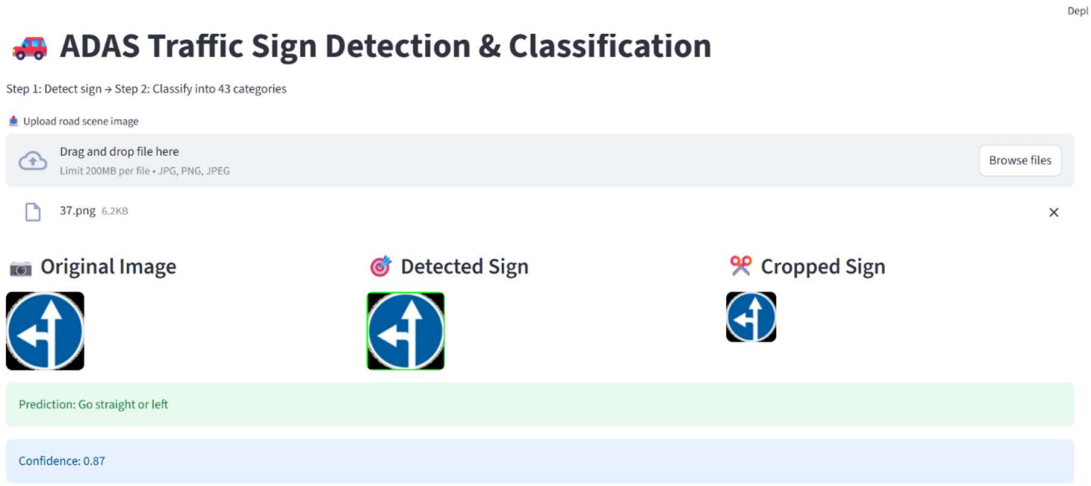


CHAPTER 11

DEPLOYMENT

The final system is deployed using Streamlit, which provides an interactive web interface for users. The application allows users to upload an image, view the detected sign, and see the classification results.

The deployment demonstrates the practical applicability of the system and provides a user-friendly way to interact with the model.



CONCLUSION

In this project, a comprehensive and practical solution for traffic sign detection and classification within an ADAS framework has been successfully developed and implemented. Unlike conventional approaches that focus solely on classification using pre-cropped images, this project addresses the more complex and realistic problem of detecting traffic signs directly from full road scene images and then classifying them accurately. The system was designed as a two-stage pipeline, where the detection stage isolates the region of interest using a combination of HSV-based color segmentation, contour filtering, and fallback mechanisms such as YOLO-based object detection. This multi-layered detection strategy significantly improves robustness and ensures that the system can handle a wide variety of real-world scenarios, including challenging lighting conditions and partial occlusions. The classification stage utilized both a custom CNN architecture and transfer learning models, including MobileNetV2 and ResNet50. Through extensive experimentation, it was observed that transfer learning models significantly outperform traditional CNNs in terms of accuracy due to their ability to leverage pre-trained feature representations. Among these, ResNet50 achieved the highest classification accuracy, making it the most suitable candidate for scenarios where precision is critical. A major highlight of this project is the Speed vs Accuracy Trade-off Analysis, which plays a crucial role in determining the deployment feasibility of models in real-world ADAS systems. While ResNet50 provides superior accuracy, MobileNetV2 offers faster inference times, making it more suitable for edge devices with limited computational resources. This analysis demonstrates the importance of considering both performance and efficiency when designing intelligent systems. The final system was successfully deployed using Streamlit, providing an interactive interface for users to upload images and visualize the detection and classification results. This deployment not only validates the functionality of the pipeline but also demonstrates its usability in real-world applications. Despite achieving strong performance, certain challenges were observed, particularly in detecting and classifying yellow and low-contrast signs. These limitations highlight potential areas for future improvement, such as incorporating more advanced object detection models and expanding the dataset. In conclusion, this project successfully demonstrates a real-world ADAS perception pipeline, integrating computer vision and deep learning techniques to solve a complex problem. It provides valuable insights into system design, model selection, and deployment considerations, making it a strong foundation for further research and development in intelligent transportation systems.